



**Universidade Federal do Pará – Campus Tucuruí
Faculdade de Engenharia de Computação Teoria
da Computação**

Projeto Teoria da Computação I

Docente: Otávio Noura Teixeira

Discentes: Enzo Silva dos Santos Silva, Caetano Otávio Garcia de Oliveira, Henrique Costa Fernandes

1 Introdução

Este relatório apresenta o desenvolvimento do Projeto 1, cujo objetivo é a definição de um Autômato Finito Não Determinístico (AFND), sua conversão para um Autômato Finito Determinístico (AFD) e a validação de palavras pertencentes ou não à linguagem reconhecida. O desenvolvimento foi realizado utilizando diagramas de estados e tabelas de transições, conforme apresentado em aula, permitindo uma melhor visualização e compreensão do funcionamento dos autômatos.

2 Definição da Linguagem

A linguagem considerada neste projeto é definida sobre o alfabeto $\Sigma = \{a, b, c\}$. Essa linguagem é composta por todas as palavras formadas a partir desse alfabeto que possuem, em algum ponto da cadeia, a subpalavra abc. Assim, uma palavra pertence à linguagem se contiver a sequência de símbolos 'a', seguida de 'b' e posteriormente de 'c', nessa ordem, independentemente da posição em que essa subpalavra ocorre.

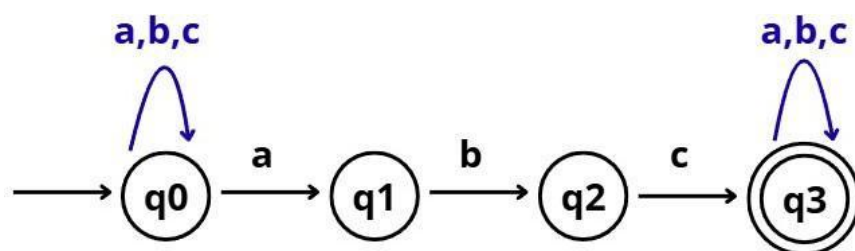
Formalmente, a linguagem pode ser expressa como:

$$L = \{w/w \text{ tem abc como subpalavra}\}, \Sigma = \{a, b, c\}$$

3 Autômato Finito Não Determinístico (AFND)

Para o reconhecimento da linguagem definida na Seção 2, foi construído um Autômato Finito Não Determinístico (AFND). Esse modelo permite a existência de múltiplas transições para um mesmo símbolo de entrada, possibilitando a representação do comportamento não determinístico necessário para o reconhecimento da subpalavra abc em qualquer posição da cadeia.

O AFND foi definido formalmente por meio de seus estados, alfabeto, função de transição, estado inicial e conjunto de estados finais. Além disso, sua estrutura é apresentada graficamente por meio de um diagrama de estados, representado na forma de um grafo dirigido, no qual os vértices correspondem aos estados do autômato e as arestas representam as transições rotuladas pelos símbolos do alfabeto.



Agora com o AFND podemos definir a máquina (M_{AFND}):

$$M_{AFND} = (\Sigma, Q, \delta, q_0, F)$$

onde:

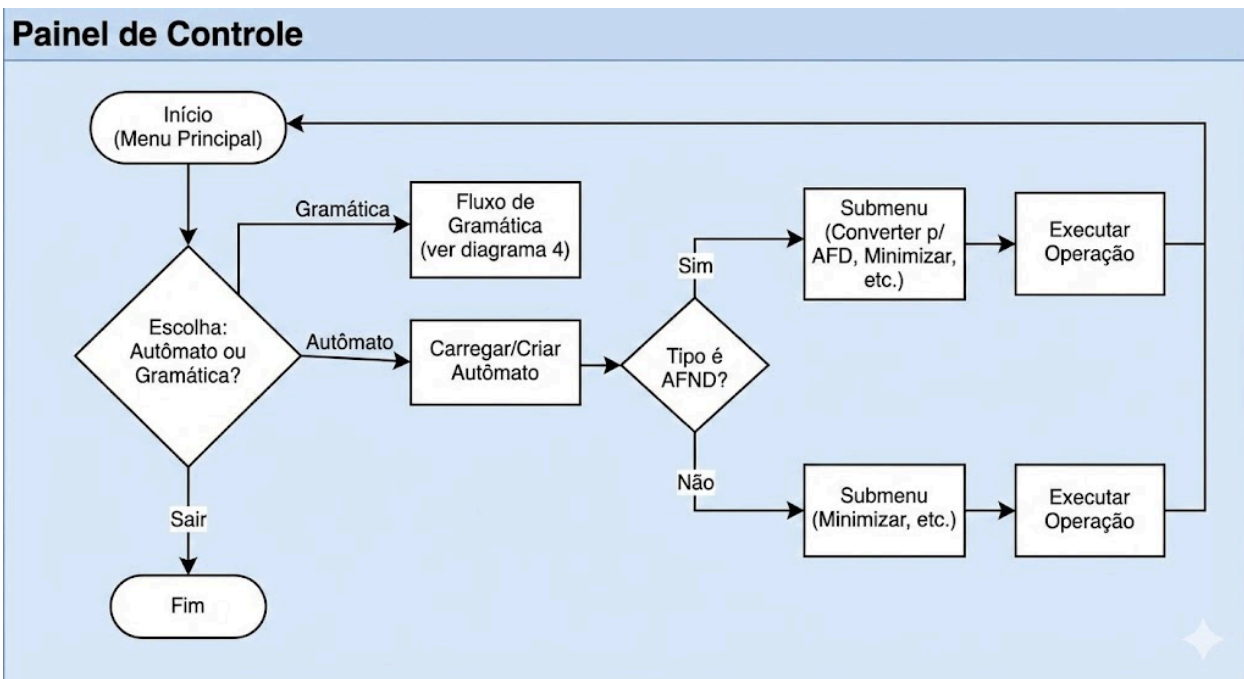
- $\Sigma = \{a, b, c\}$ é o alfabeto;
- $Q = \{q_0, q_1, q_2, q_3\}$ é o conjunto de estados;
- δ é a função de transição ou programação;
- q_0 é o estado inicial;
- $F = \{q_3\}$

Dessa forma temos que a definição formal da máquina é:

$$M_{AFND} = (\{a, b, c\}, \{q_0, q_1, q_2, q_3\}, \delta, q_0, \{q_3\})$$

A função de transição δ definimos através da tabela abaixo.

δ	a	b	c
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0\}$
q_1	-	$\{q_2\}$	-
q_2	-	-	$\{q_3\}$
$*q_3$	$\{q_3\}$	$\{q_3\}$	$\{q_3\}$



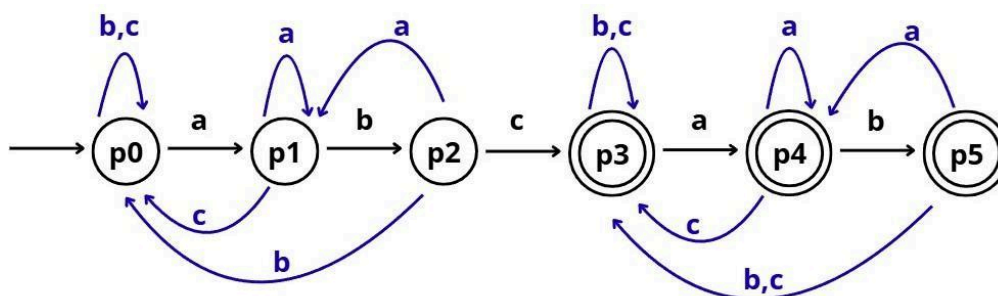
4 Conversão do AFND para AFD

[

δ'	a	b	c
$\rightarrow \langle q0 \rangle P0$	$\langle q0 q1 \rangle P1$	$\langle q0 \rangle P0$	$\langle q0 \rangle P0$
$\langle q0 q1 \rangle P1$	$\langle q0 q1 \rangle P1$	$\langle q0 q2 \rangle P2$	$\langle q0 \rangle P0$
$\langle q0 q2 \rangle P2$	$\langle q0 q1 \rangle P1$	$\langle q0 \rangle P0$	$\langle q0 q3 \rangle P3$
$\langle q0 q3 \rangle *P3$	$\langle q0 q1 q3 \rangle P4$	$\langle q0 q3 \rangle P3$	$\langle q0 q3 \rangle P3$
$\langle q0 q1 q3 \rangle *P4$	$\langle q0 q1 q3 \rangle P4$	$\langle q0 q2 q3 \rangle P5$	$\langle q0 q3 \rangle P3$
$\langle q0 q2 q3 \rangle *P5$	$\langle q0 q1 q3 \rangle P4$	$\langle q0 q3 \rangle P3$	$\langle q0 q3 \rangle P3$

A conversão do Autômato Finito Não Determinístico (AFND) para Autômato Finito Determinístico (AFD) foi realizada por meio do método do subconjunto, no qual cada estado do AFD corresponde a um subconjunto de estados do AFND. A partir do estado inicial do AFND, foram gerados os novos estados e definidas as transições necessárias para a construção do AFD equivalente

Agora com a função de transição do AFD, feita acima, foi realizada a construção do grafo equivalente ao AFD e em seguida foi feita a máquina (M_{AFD}) e a validação de palavras.



$$M_{AFD} = (\{a, b, c\}, \{p0, p1, p2, p3, p4, p5\}, \delta', p0, \{p3, p4, p5\})$$

validação:

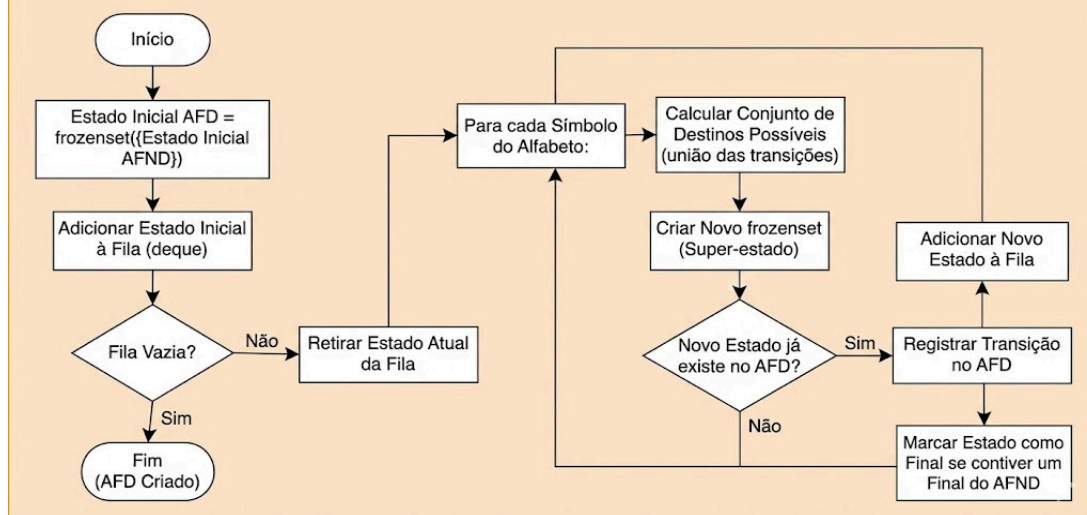
w = cabca

$\rightarrow p_0 \rightarrow p_0 \rightarrow p_1 \rightarrow p_2 \rightarrow p_3 \rightarrow p_4 \in F \checkmark$ (palavra aceita)

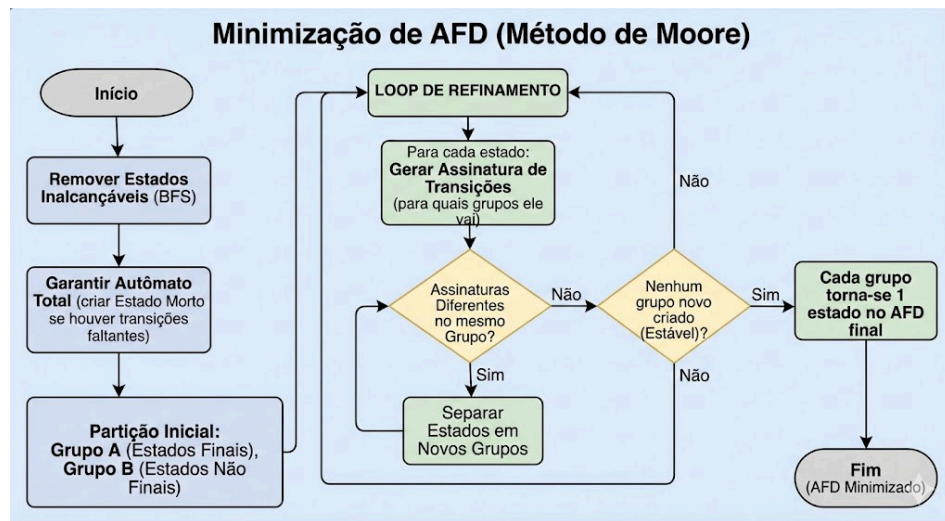
w = acbac

$\rightarrow p_0 \rightarrow p_0 \rightarrow p_0 \rightarrow p_0 \rightarrow p_1 \rightarrow p_0 \notin F \times$ (palavra não aceita)

Conversão: AFND para AFD



5 Minimização de AFD



Após a determinação, o sistema permite minimizar o AFD gerado. O processo de minimização é dividido em três etapas principais. Na minimização foi usado o método de Moore:

5.1 Remoção de Estados Inalcançáveis

Inicialmente, é feita uma busca a partir do estado inicial para identificar apenas os estados que realmente podem ser atingidos. Estados inalcançáveis são removidos.

5.2 Completação com Estado Morto

Caso existam transições não definidas para algum símbolo do alfabeto, o sistema cria um estado morto, garantindo que o autômato se torne completo. O estado morto possui transições para si mesmo em todos os símbolos.

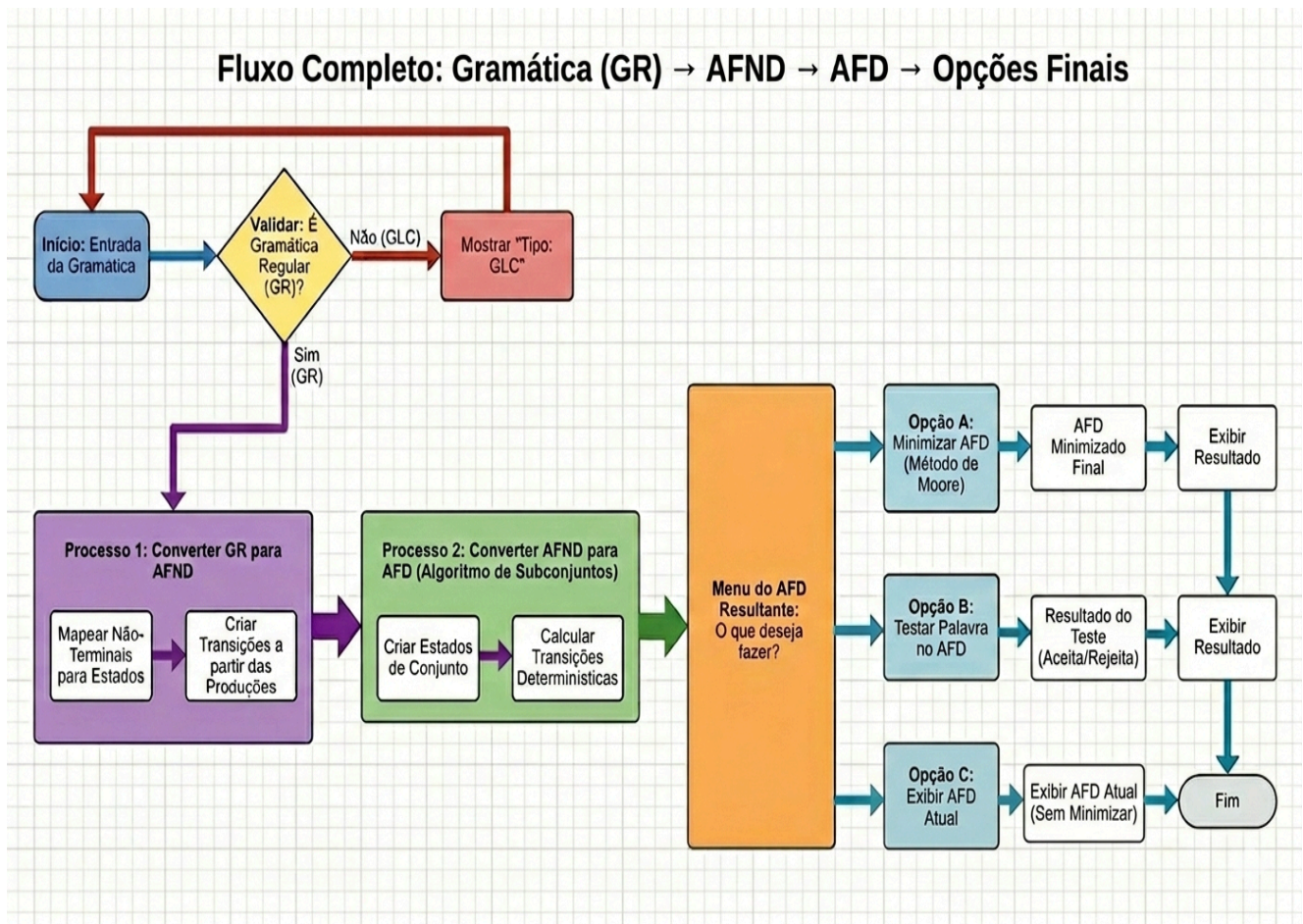
5.3 Particionamento de Estados

O algoritmo separa inicialmente os estados em dois grupos: Estados finais e Estados não finais. Em seguida, aplica refinamentos sucessivos, dividindo grupos até que não seja mais possível separar estados com comportamentos distintos. Estados que permanecem no mesmo grupo ao final do processo são considerados equivalentes. O resultado é um AFD minimizado, com menor número possível de estados, mas reconhecendo a mesma linguagem.

Link da conversa com o Deepseek sobre a minimização:

<https://chat.deepseek.com/share/vkm6drmxjkqumoni46>

6. Manipulação de Gramáticas Regulares



O sistema também permite criar e analisar gramáticas.

“Formalmente, uma gramática é definida como:

$$G = (V, \Sigma, P, S)$$

Onde:

- V é o conjunto de variáveis ou não-terminais
- Σ é o conjunto de símbolos terminais
- P é o conjunto de produções

- S é o símbolo inicial

No sistema, o usuário informa esses quatro elementos.”

6.1 Verificação do Tipo da Gramática

As produções inseridas pelo usuário são analisadas para verificar se seguem o padrão de gramática regular.

No nosso sistema, as produções precisam estar no formato:

$A \rightarrow aB$

$A \rightarrow a$

$A \rightarrow \varepsilon$

Caso não sigam o padrão da Gramática Regular o sistema o classifica como Gramática Livre de Contexto (GLC).

6.2 Conversão de Gramática Regular para AFND

A conversão segue as regras clássicas:

- $A \rightarrow aB$ gera uma transição do estado A para B com símbolo a.
- $A \rightarrow a$ gera transição para um estado final auxiliar.
- $A \rightarrow \varepsilon$ transforma o estado A em estado final.

É criado um estado final adicional para representar produções terminais simples.

6.3 Conversão de AFND para AFD

Após a construção do AFND a partir da Gramática Regular, o sistema realiza a conversão para Autômato Finito Determinístico (AFD), utilizando o **Algoritmo de Subconjuntos**.

Essa etapa é necessária porque o AFND pode possuir:

- Mais de uma transição para o mesmo símbolo;
- Múltiplos estados possíveis após a leitura de um símbolo;
- Comportamento não determinístico.

O AFD elimina essa ambiguidade, garantindo que:

Para cada estado e para cada símbolo do alfabeto exista exatamente uma única transição.

Funcionamento da Conversão

O processo ocorre da seguinte forma:

1. Definição do estado inicial do AFD

O estado inicial do AFD é formado pelo conjunto contendo apenas o estado inicial do AFND.

Exemplo:

Se o estado inicial do AFND é q_0 ,
então o estado inicial do AFD será $\{q_0\}$.

2. Criação de estados como conjuntos

Cada estado do AFD representa um conjunto de estados do AFND.

Para cada estado-conjunto criado e para cada símbolo do alfabeto:

- São verificadas todas as transições possíveis no AFND;
- É feita a união dos estados alcançados;
- Esse novo conjunto passa a representar um novo estado do AFD.

Caso esse conjunto ainda não exista, ele é adicionado à lista de estados.

3. Determinação das transições

As transições do AFD são construídas associando:

Estado-conjunto + símbolo $\rightarrow$ novo estado-conjunto resultante.

Dessa forma, o comportamento torna-se totalmente determinístico.

4. Definição dos estados finais

Um estado do AFD será considerado final se:

Ele contiver pelo menos um estado final do AFND.

Ou seja, se qualquer estado dentro do conjunto for final no AFND, todo o conjunto será marcado como estado final no AFD.

6.4 Operações sobre o AFD

Após a construção do AFD, o sistema permite três operações principais:

a) Minimização do AFD

O sistema aplica o método de refinamento de partições (Método de Moore) para reduzir o número de estados, mantendo a equivalência da linguagem reconhecida.

O objetivo é:

- Eliminar estados equivalentes;
- Produzir um AFD mínimo;
- Manter a mesma linguagem reconhecida.

7. Teste de Palavras

O sistema permite testar se uma palavra pertence à linguagem reconhecida pelo autômato.

No caso de AFD:

- A palavra é processada símbolo por símbolo;
- O estado atual é atualizado deterministicamente;
- Ao final, verifica-se se o estado alcançado é final

No caso de AFND:

- Mantém-se um conjunto de estados atuais;

- Para cada símbolo, calcula-se o conjunto de todos os possíveis estados seguintes;
- Se ao final houver interseção com os estados finais, a palavra é aceita.

Considerações Finais

O desenvolvimento deste trabalho foi fundamental para compreender de forma prática os conceitos estudados em Teoria da Computação. Ao implementar o sistema de autômatos e gramáticas, foi possível visualizar na prática como funcionam os modelos formais responsáveis pelo reconhecimento e geração de linguagens.

Durante a construção do projeto, ficou evidente a diferença entre Autômatos Finitos Determinísticos e Não Determinísticos, bem como a equivalência entre eles. A implementação da conversão de AFND para AFD permitiu entender melhor o método do subconjunto e perceber que, apesar das diferenças estruturais, ambos reconhecem o mesmo tipo de linguagem. A etapa de minimização também contribuiu para compreender como é possível otimizar um autômato sem alterar a linguagem reconhecida.

A parte relacionada às Gramáticas Regulares foi importante para reforçar a ideia de que uma linguagem pode ser representada tanto por regras de produção quanto por um autômato finito. A conversão automática da gramática para AFND e, posteriormente, para AFD mostrou na prática essa relação teórica estudada em sala de aula.

De forma geral, o projeto ajudou a consolidar os conteúdos aprendidos, tornando mais claro o funcionamento dos modelos formais e sua aplicação. Além disso, a organização do sistema permite que ele seja expandido futuramente, incluindo novas funcionalidades e aprofundando ainda mais o estudo das linguagens formais.